CHAPTER 9

Type Safety, Proved

In Chapter 1 we quoted Robin Milner's slogan, *well-typed programs cannot go wrong*, and split it into two halves: **progress** (a well-typed program is never stuck) and **preservation** (a step of computation does not change the type). Chapter 3 stated both for the small language `Arith`, and Chapter 8 built the simply typed λ-calculus. All along, the theorem has been a promise. In this chapter we keep it.

We take the simply typed λ-calculus with booleans and natural numbers, define how its programs run, one small step at a time, and prove in Lean that no well-typed program ever reaches a stuck state, no matter how long it runs; we also prove that evaluation is deterministic. The proof is the classic one, due in this form to Andrew Wright and Matthias Felleisen (1994) and presented in almost every textbook on programming languages since (Pierce's *Types and Programming Languages* is the standard reference). It is built from half a dozen lemmas, and we will go through every one of them, line by line. If you have never read a proof about a programming language, this chapter shows you how one is put together and where the real difficulty hides.

Then we look at the same theorems from the other side: a Rust crate generates thousands of random well-typed programs and runs them, checking progress and preservation at every step. Putting tests and proofs side by side is the best way to understand what each one gives you. Finally we ask what "going wrong" means for Rust itself.

In this chapter you will learn:

- how to define a small-step operational semantics as an inductive relation, and what values and stuck terms are;
- the canonical forms lemmas, and how they make the progress theorem work;
- why preservation needs weakening and substitution lemmas, and how de Bruijn indices make those lemmas precise enough for a machine to check;
- how progress and preservation combine into type safety for runs of any length, and why evaluation is deterministic;
- how to test the same theorems with a property-based test harness in Rust, and what such tests can and cannot tell you;
- what "going wrong" means for Rust, and what its type system does not promise.

9.1 The language

The language of this chapter is the simply typed λ-calculus of Chapter 8, cut down to what the proofs need: functions, booleans and natural numbers.

$$\tau ::= \mathit{bool} \mid \mathit{nat} \mid \tau_1 \to \tau_2$$

$$t ::= \#i \mid \lambda : \tau . t \mid t_1\, t_2 \mid \mathit{true} \mid \mathit{false} \mid \mathit{if}\ t_1\ \mathit{then}\ t_2\ \mathit{else}\ t_3 \mid n \mid t_1 + t_2 \mid \mathit{iszero}\ t$$

Variables are **de Bruijn indices**, as in Chapter 7: instead of a name, a variable is the number of binders between it and the λ that binds it. We write them $\#0$, $\#1$, $\#2$, with a `#` so that they cannot be confused with the numeric literals $0, 1, 2$ of the language. The function $\lambda(x : \mathit{nat}).\, x + 1$ becomes $\lambda : \mathit{nat}.\, \#0 + 1$, and the function that applies its first argument twice to its second, $\lambda(f : \mathit{nat} \to \mathit{nat}).\, \lambda(x : \mathit{nat}).\, f\,(f\,x)$, becomes $\lambda : (\mathit{nat} \to \mathit{nat}).\, \lambda : \mathit{nat}.\, \#1\,(\#1\,\#0)$: inside the inner λ, x is one binder away ($\#0$) and f is two ($\#1$).

Why de Bruijn indices, when names are so much easier to read? Because the proof of preservation hinges on a lemma about substitution, and with names, substitution must avoid capturing variables, terms must be identified up to renaming of bound variables (α-equivalence), and every lemma acquires side conditions about fresh names. On paper these are usually waved away. In a proof assistant nothing can be waved away, and de Bruijn indices make all of it disappear: there are no names to clash, and two α-equivalent terms are literally equal. The price is that indices must be adjusted when a term moves under a binder, and we will pay that price, carefully, when we get to substitution.

The λ carries a type annotation, $\lambda : \tau . t$: the type of the parameter is written down, not inferred. Chapter 8's bidirectional checker could sometimes do without it; here every λ has one, so that type checking is a simple recursive function and every term has at most one type.

In Lean

The language lives in `code/lean/Book/Ch09/Syntax.lean`. Types and terms are two inductive types:

```
/-- The types of the language: booleans, natural numbers and functions. -/
inductive Ty where
  | bool : Ty
  | nat : Ty
  | arrow : Ty → Ty → Ty
  deriving DecidableEq
```

```
/-- Terms in de Bruijn notation: `var i` refers to the binder `i` levels up.
`lam τ t` is `λ (x : τ). t`, with the bound variable written `var 0` in `t`.
`tt` and `ff` are the boolean literals (`true` and `false` are Lean's own). -/
inductive Term where
  | var : Nat → Term
  | lam : Ty → Term → Term
  | app : Term → Term → Term
  | tt : Term
  | ff : Term
  | ite : Term → Term → Term → Term
  | lit : Nat → Term
  | add : Term → Term → Term
  | isZero : Term → Term
  deriving DecidableEq
```

The constructor `ite` is "if-then-else" (the name Lean itself uses for `if`, which is a keyword), and the boolean literals are `tt` and `ff`, because `true` and `false` already name Lean's own booleans. The file also defines a printer, so that `#eval` shows terms in the notation of this chapter:

```
/-- `(λ (x : nat). x + 1) 41` -/
def incr41 : Term := .app (.lam .nat (.add (.var 0) (.lit 1))) (.lit 41)

/-- `λ (f : nat → nat). λ (x : nat). f (f x)`: apply a function twice. The inner
`#1` is `f`, one binder further out than `x`, which is `#0`. -/
def twice : Term :=
  .lam (.arrow .nat .nat) (.lam .nat (.app (.var 1) (.app (.var 1) (.var 0))))

/-- info: (λ:nat. #0 + 1) 41 -/
#guard_msgs in
#eval incr41

/-- info: λ:(nat → nat). λ:nat. #1 (#1 #0) -/
#guard_msgs in
#eval twice
```

There is one important difference from Chapter 8. There, terms were **intrinsically** typed: the type of a term was an index of its Lean type (`Term Γ τ`), so an ill-typed term could not even be written down, and the evaluator was safe by construction. Here terms are **extrinsically** typed: `Term` is plain syntax, ill-typed terms such as `true + 1` are perfectly good values of type `Term`, and typing is a separate relation that some terms satisfy and others do not. The intrinsic approach gives safety for free; the extrinsic approach makes safety a *theorem*, something we can state, prove and break. To talk about programs that go wrong, we need to be able to write them.

In Rust

The Rust crate `safety` (`code/rust/crates/safety`) mirrors the Lean files module by module. Its types and terms are the same two definitions, as Rust enums:

```
/// The types: booleans, natural numbers and functions.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Ty {
    Bool,
    Nat,
    Arrow(Box<Ty>, Box<Ty>),
}
```

```
/// Terms in de Bruijn notation. `Var(i)` refers to the binder `i` levels up;
/// `Lam(τ, t)` is `λ (x : τ). t`, with the bound variable written `Var(0)` in `t`.
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Term {
    Var(usize),
    Lam(Ty, Box<Term>),
    App(Box<Term>, Box<Term>),
    True,
    False,
    If(Box<Term>, Box<Term>, Box<Term>),
    Lit(u64),
    Add(Box<Term>, Box<Term>),
    IsZero(Box<Term>),
}
```

The Rust printer produces exactly the same notation as the Lean one, so the outputs of the two languages can be compared character by character.

9.2 The typing rules

A context Γ is a list of types: entry i is the type of the variable $\#i$. The variable bound by the innermost λ is $\#0$, so when the type checker enters a $\lambda : \sigma$, it puts σ at the *front* of the list, and every variable that was already in scope moves one position back, exactly as its index does. We write $\sigma :: \Gamma$ for that extended context and $\Gamma(i)$ for entry i. The rules are those of Chapter 8, restricted to our types:

$$\frac{\Gamma(i) = \tau}{\Gamma \vdash \#i : \tau}\ \text{T-Var} \qquad \frac{\sigma :: \Gamma \vdash t : \tau}{\Gamma \vdash \lambda : \sigma.\, t : \sigma \to \tau}\ \text{T-Lam}$$

$$\frac{\Gamma \vdash t_1 : \sigma \to \tau \qquad \Gamma \vdash t_2 : \sigma}{\Gamma \vdash t_1\ t_2 : \tau}\ \text{T-App}$$

$$\frac{}{\Gamma \vdash \mathit{true} : \mathit{bool}}\ \text{T-True} \qquad \frac{}{\Gamma \vdash \mathit{false} : \mathit{bool}}\ \text{T-False} \qquad \frac{}{\Gamma \vdash n : \mathit{nat}}\ \text{T-Lit}$$

$$\frac{\Gamma \vdash t_1 : \mathit{bool} \qquad \Gamma \vdash t_2 : \tau \qquad \Gamma \vdash t_3 : \tau}{\Gamma \vdash \mathit{if}\ t_1\ \mathit{then}\ t_2\ \mathit{else}\ t_3 : \tau}\ \text{T-If}$$

$$\frac{\Gamma \vdash t_1 : \mathit{nat} \qquad \Gamma \vdash t_2 : \mathit{nat}}{\Gamma \vdash t_1 + t_2 : \mathit{nat}}\ \text{T-Add} \qquad \frac{\Gamma \vdash t : \mathit{nat}}{\Gamma \vdash \mathit{iszero}\ t : \mathit{bool}}\ \text{T-IsZero}$$

Rule T-If is the rule from Chapter 1 that both branches share one type; it will come back later in the chapter, when we break it on purpose.

In Lean, contexts are lists, `lookup` is $\Gamma(i)$, and the judgment is an inductive predicate with one constructor per rule, the technique of Chapter 3 (file `code/lean/Book/Ch09/Typing.lean`):

```
/-- A context lists the types of the free variables: entry `i` is the type of
`var i`. The most recently bound variable comes first. -/
abbrev Ctx := List Ty

/-- The type of variable `i` in `Γ`, if `Γ` has an entry `i`. -/
def lookup : Ctx → Nat → Option Ty
  | [], _ => none
  | σ :: _, 0 => some σ
  | _ :: Γ, i + 1 => lookup Γ i
```

```
/-- `HasType Γ t τ`: in context `Γ`, the term `t` has type `τ`. -/
inductive HasType : Ctx → Term → Ty → Prop
  | var {Γ : Ctx} {i : Nat} {τ : Ty} :
      lookup Γ i = some τ → HasType Γ (.var i) τ
  | lam {Γ : Ctx} {σ τ : Ty} {t : Term} :
      HasType (σ :: Γ) t τ → HasType Γ (.lam σ t) (.arrow σ τ)
  | app {Γ : Ctx} {σ τ : Ty} {t₁ t₂ : Term} :
      HasType Γ t₁ (.arrow σ τ) → HasType Γ t₂ σ → HasType Γ (.app t₁ t₂) τ
  | tt {Γ : Ctx} : HasType Γ .tt .bool
  | ff {Γ : Ctx} : HasType Γ .ff .bool
  | ite {Γ : Ctx} {τ : Ty} {c t e : Term} :
      HasType Γ c .bool → HasType Γ t τ → HasType Γ e τ →
      HasType Γ (.ite c t e) τ
  | lit {Γ : Ctx} {n : Nat} : HasType Γ (.lit n) .nat
  | add {Γ : Ctx} {t₁ t₂ : Term} :
      HasType Γ t₁ .nat → HasType Γ t₂ .nat → HasType Γ (.add t₁ t₂) .nat
  | isZero {Γ : Ctx} {t : Term} :
      HasType Γ t .nat → HasType Γ (.isZero t) .bool
```

Compare each constructor with its rule: the hypotheses before the last arrow are the premises, the type after it is the conclusion, and the variables in curly braces are the metavariables of the rule, which Lean will fill in by itself whenever it can. (We wrote our own `lookup` rather than using Lean's

list indexing `Γ[i]?`. The two compute the same thing, but `lookup` is three lines that you can read, and it keeps the whole development free of axioms, a point we will come back to at the end.)

A derivation is a value of this inductive type. Here is the typing derivation for `twice`, built by hand:

```
/-- A derivation, built by hand: `twice` has type `(nat → nat) → nat → nat`.
Read the proof term from the outside in: two `lam` rules, then the outer
application, whose function is `#1` and whose argument is another application. -/
example : HasType [] twice (.arrow (.arrow .nat .nat) (.arrow .nat .nat)) :=
  .lam (.lam
    (.app (.var rfl)                          -- #1 : nat → nat
      (.app (.var rfl) (.var rfl))))          -- #1 #0 : nat
```

Read it as the derivation tree written sideways. The outermost `.lam` is T-Lam for $\lambda : (\mathit{nat} \to \mathit{nat})$, the next one for $\lambda : \mathit{nat}$, and below them the body $\#1\ (\#1\ \#0)$ is an application (T-App) whose function is the variable $\#1$ and whose argument is another application. Each `.var rfl` is T-Var: its premise is an equation such as `lookup [nat, nat → nat] 1 = some (nat → nat)`, and `rfl` proves it because both sides compute to the same value. Lean worked out every type in the tree from the conclusion we asked for.

A checker, proved sound

Building derivations by hand gets tedious quickly. As in Chapter 3, a type checker does it for us. The checker `typeOf` has one case per rule:

```
/-- The type checker: `some τ` if the term has type `τ`, `none` if it has no
type. -/
def typeOf (Γ : Ctx) : Term → Option Ty
  | .var i => lookup Γ i
  | .lam σ t =>
    match typeOf (σ :: Γ) t with
    | some τ => some (.arrow σ τ)
    | none => none
  | .app t₁ t₂ =>
    match typeOf Γ t₁, typeOf Γ t₂ with
    | some (.arrow σ τ), some σ' => if σ = σ' then some τ else none
    | _, _ => none
  | .tt => some .bool
  | .ff => some .bool
  | .ite c t e =>
    match typeOf Γ c, typeOf Γ t, typeOf Γ e with
    | some .bool, some τ, some τ' => if τ = τ' then some τ else none
    | _, _, _ => none
  | .lit _ => some .nat
  | .add t₁ t₂ =>
    match typeOf Γ t₁, typeOf Γ t₂ with
    | some .nat, some .nat => some .nat
    | _, _ => none
  | .isZero t =>
    match typeOf Γ t with
    | some .nat => some .bool
    | _ => none
```

```
/-- info: some ((nat → nat) → nat → nat) -/
#guard_msgs in
#eval typeOf [] twice

/-- info: none -/
#guard_msgs in
#eval typeOf [] (.add .tt (.lit 1))
```

and it is proved sound with respect to the rules: whenever it says `some τ`, a derivation of type `τ` exists. The proof goes by induction on the term, with the same technique as Chapter 3; here are the first cases:

```
/-- The checker is sound: when it answers `some τ`, a derivation exists. -/
theorem typeOf_sound {Γ : Ctx} {t : Term} {τ : Ty} :
    typeOf Γ t = some τ → HasType Γ t τ := by
  induction t generalizing Γ τ with
  | var i => intro h; exact .var h
  | lam σ t ih =>
    intro h
    simp only [typeOf] at h
    split at h
    · next τ' h' => cases h; exact .lam (ih h')
    · cases h
  | app t₁ t₂ ih₁ ih₂ =>
    intro h
    simp only [typeOf] at h
    split at h
    · next σ τ' σ' h₁ h₂ =>
      split at h
      · next hσ => cases h; subst hσ; exact .app (ih₁ h₁) (ih₂ h₂)
      · cases h
    · cases h
```

In each case with subterms, `simp only [typeOf] at h` unfolds one step of the checker in the hypothesis `h : typeOf Γ t = some τ`, and `split at h` splits the proof into one branch per arm of the checker's `match`; `next ... =>` names the hypotheses the branch provides. Where the checker returns `none`, `h` has become `none = some τ`, which `cases h` dismisses (no constructor of `=` can prove it). In the successful branch, `cases h` learns what `τ` is, and the derivation is assembled from the induction hypotheses. The remaining cases, in the file, are the same. With a sound checker, the typing derivation of a concrete program is just a computation:

```
/-- With a sound checker, a derivation for a concrete program is a computation:
`rfl` checks that the checker says `some nat`. -/
theorem incr41_typed : HasType [] incr41 .nat := typeOf_sound rfl
```

The Rust checker is the same function, returning a `Result` with an error message instead of an `Option`, and with a switch, `rules`, that we will use later to break rule T-If:

```
/// The type of `t` in context `ctx`, under the given rules.
pub fn type_of_with(rules: Rules, ctx: &[Ty], t: &Term) -> Result<Ty, TypeError> {
    let expect = |place, u: &Term, expected: Ty| -> Result<(), TypeError> {
        let found = type_of_with(rules, ctx, u)?;
        if found == expected {
            Ok(())
        } else {
            Err(TypeError::Mismatch {
                place,
                expected,
                found,
            })
        }
    };
    match t {
        // T-Var: look the variable up in the context.
        Term::Var(i) => ctx.get(*i).cloned().ok_or(TypeError::UnboundVariable(*i)),
        // T-Lam: check the body in the context extended with the parameter.
        Term::Lam(param, body) => {
            let mut inner = vec![param.clone()];
            inner.extend_from_slice(ctx);
```

```
            let result = type_of_with(rules, &inner, body)?;
            Ok(Ty::arrow(param.clone(), result))
        }
        // T-App: a function of type σ → τ applied to an argument of type σ.
        Term::App(f, a) => match type_of_with(rules, ctx, f)? {
            Ty::Arrow(param, result) => {
                expect("the argument", a, *param)?;
                Ok(*result)
            }
            other => Err(TypeError::NotAFunction(other)),
        },
        // T-True, T-False, T-Lit.
        Term::True | Term::False => Ok(Ty::Bool),
        Term::Lit(_) => Ok(Ty::Nat),
        // T-If: a boolean condition and two branches of the same type.
        Term::If(c, then_branch, else_branch) => {
            expect("the condition of `if`", c, Ty::Bool)?;
            let ty_then = type_of_with(rules, ctx, then_branch)?;
            let ty_else = type_of_with(rules, ctx, else_branch)?;
            match rules {
                Rules::Sound if ty_then != ty_else => {
                    Err(TypeError::BranchesDiffer(ty_then, ty_else))
                }
                // The broken rule never compares the two branches.
                _ => Ok(ty_then),
            }
        }
        // T-Add and T-IsZero.
        Term::Add(a, b) => {
            expect("the left operand of `+`", a, Ty::Nat)?;
            expect("the right operand of `+`", b, Ty::Nat)?;
            Ok(Ty::Nat)
        }
        Term::IsZero(n) => {
            expect("the argument of `iszero`", n, Ty::Nat)?;
            Ok(Ty::Bool)
        }
    }
}
```

9.3 Running programs: small-step semantics

A type system is a claim about how programs behave, so before we can prove anything we must say precisely how programs run. We do it with a **small-step operational semantics**: a relation $t \to t'$ that says "the term t takes one step of computation and becomes t'". A whole run is a chain of such steps.

Values

Some terms are finished: they are the possible results of a computation. These are the **values**:

$$v ::= \lambda : \tau . t \mid \mathit{true} \mid \mathit{false} \mid n$$

A λ-abstraction is a value even if its body could compute further: we never evaluate under a binder, just as Rust does not run the body of a closure until the closure is called. In Lean, values are an inductive predicate on terms (file `code/lean/Book/Ch09/Semantics.lean`):

```
/-- Values: the terms that are finished computing. -/
inductive Value : Term → Prop
  | lam {τ : Ty} {t : Term} : Value (.lam τ t)
  | tt : Value .tt
  | ff : Value .ff
  | lit {n : Nat} : Value (.lit n)
```

The evaluation rules

The rules come in two kinds. **Computation rules** do the actual work: they apply when a term is a redex, an elimination form applied to a value of the right shape.

$$\frac{v \text{ value}}{(\lambda : \tau.\, t)\, v \to t[v]}\ \text{E-Beta}$$

$$\frac{}{\mathit{if}\ \mathit{true}\ \mathit{then}\ t_2\ \mathit{else}\ t_3 \to t_2}\ \text{E-IfTrue} \qquad \frac{}{\mathit{if}\ \mathit{false}\ \mathit{then}\ t_2\ \mathit{else}\ t_3 \to t_3}\ \text{E-IfFalse}$$

$$\frac{}{m + n \to m + n}\ \text{E-AddLit} \qquad \frac{}{\mathit{iszero}\ 0 \to \mathit{true}}\ \text{E-IsZeroZero} \qquad \frac{}{\mathit{iszero}\ (n+1) \to \mathit{false}}\ \text{E-IsZeroSucc}$$

In E-AddLit, m and n are literals and $m + n$ on the right is the literal for their sum: the step turns the *expression* $2 + 3$ into the *number* 5. In E-Beta, $t[v]$ is the body t with its bound variable $\#0$ replaced by v; we will define it precisely in the section on substitution.

Congruence rules say where to look for the next redex when the term is not one yet:

$$\frac{t_1 \to t_1'}{t_1\ t_2 \to t_1'\ t_2}\ \text{E-AppL} \qquad \frac{v_1 \text{ value} \qquad t_2 \to t_2'}{v_1\ t_2 \to v_1\ t_2'}\ \text{E-AppR}$$

$$\frac{t_1 \to t_1'}{\mathit{if}\ t_1\ \mathit{then}\ t_2\ \mathit{else}\ t_3 \to \mathit{if}\ t_1'\ \mathit{then}\ t_2\ \mathit{else}\ t_3}\ \text{E-If}$$

$$\frac{t_1 \to t_1'}{t_1 + t_2 \to t_1' + t_2}\ \text{E-AddL} \qquad \frac{v_1 \text{ value} \qquad t_2 \to t_2'}{v_1 + t_2 \to v_1 + t_2'}\ \text{E-AddR} \qquad \frac{t \to t'}{\mathit{iszero}\ t \to \mathit{iszero}\ t'}\ \text{E-IsZero}$$

Together these rules fix an **evaluation strategy**, call-by-value with left-to-right order, the strategy of Chapter 7 and of Rust. The premise "v_1 value" in E-AppR and E-AddR is what imposes the order: the right operand may only step once the left one is finished. The premise "v value" in E-Beta says that arguments are evaluated before the call. Rust behaves the same way: in `f(a(), b())`, the function expression is evaluated first, then `a()`, then `b()`, and only then is `f` called. Notice also what the rules do *not* say: no rule evaluates the branches of an `if` before the condition is known, and no rule looks inside a λ.

The rules transcribe into one inductive predicate, one constructor per rule:

```
/-- One step of call-by-value evaluation. The *congruence* rules (`appL`, `appR`,
`ite`, `addL`, `addR`, `isZero`) say where to look for work; the *computation*
rules (`beta`, `iteTrue`, `iteFalse`, `addLit`, `isZeroZero`, `isZeroSucc`) do
the work. -/
inductive Step : Term → Term → Prop
  | appL {t₁ t₁' t₂ : Term} :
      Step t₁ t₁' → Step (.app t₁ t₂) (.app t₁' t₂)
  | appR {v₁ t₂ t₂' : Term} :
      Value v₁ → Step t₂ t₂' → Step (.app v₁ t₂) (.app v₁ t₂')
  | beta {τ : Ty} {body v : Term} :
      Value v → Step (.app (.lam τ body) v) (instantiate body v)
  | ite {c c' t e : Term} :
      Step c c' → Step (.ite c t e) (.ite c' t e)
  | iteTrue {t e : Term} : Step (.ite .tt t e) t
  | iteFalse {t e : Term} : Step (.ite .ff t e) e
  | addL {t₁ t₁' t₂ : Term} :
      Step t₁ t₁' → Step (.add t₁ t₂) (.add t₁' t₂)
  | addR {v₁ t₂ t₂' : Term} :
      Value v₁ → Step t₂ t₂' → Step (.add v₁ t₂) (.add v₁ t₂')
  | addLit {m n : Nat} : Step (.add (.lit m) (.lit n)) (.lit (m + n))
  | isZero {t t' : Term} : Step t t' → Step (.isZero t) (.isZero t')
```

```
  | isZeroZero : Step (.isZero (.lit 0)) .tt
  | isZeroSucc {n : Nat} : Step (.isZero (.lit (n + 1))) .ff
```

Runs

A run is a sequence of steps. Its relation, written $t \xrightarrow{*} t'$, is the **reflexive-transitive closure** of $\to$: zero steps, or one step followed by a run.

```
/-- Zero or more steps: the reflexive-transitive closure of `Step`. -/
inductive Steps : Term → Term → Prop
  | refl {t : Term} : Steps t t
  | step {t t' t'' : Term} : Step t t' → Steps t' t'' → Steps t t''
```

```
/-- Two runs, one after the other, form a run. -/
theorem Steps.trans {t₁ t₂ t₃ : Term} (h₁₂ : Steps t₁ t₂) (h₂₃ : Steps t₂ t₃) :
    Steps t₁ t₃ := by
  induction h₁₂ with
  | refl => exact h₂₃
  | step hstep _ ih => exact .step hstep (ih h₂₃)
```

`Steps.trans` glues two runs together, by induction on the first run. If it is empty, the second run is already the answer. If it starts with a step `hstep`, the induction hypothesis `ih` glues the rest of it to the second run, and `.step hstep` puts the first step back in front.

Here is a complete run, written as a proof:

```
/-- `(λ:nat. #0 + 1) 41` takes two steps to reach `42`. Each step is a rule
instance; `.step` chains them and `.refl` ends the run. -/
example : Steps incr41 (.lit 42) :=
  .step (.beta .lit)            -- (λ:nat. #0 + 1) 41  →  41 + 1
    (.step .addLit              -- 41 + 1              →  42
      .refl)
```

The first step is E-Beta, whose premise is that the argument `41` is a value (`.lit` is the constructor of `Value` for literals). It produces `instantiate (#0 + 1) 41`, which Lean computes to `41 + 1`. The second step is E-AddLit, which produces the literal `lit (41 + 1)`. Lean computes `41 + 1` to `42`, so the run can end there with `.refl`: the statement asked for `.lit 42`, and that is where we are.

> **NOTE**
>
> Mathlib has the reflexive-transitive closure of any relation, `Relation.ReflTransGen`. It adds steps at the *end* of a run instead of at the front, but it is the same relation, as `steps_iff_reflTransGen` at the end of `Semantics.lean` proves. We defined our own because two constructors are easier to read than a general library, and because building it is the lesson.

Stuck terms

What happens to `true + 1`? It is not a value, and no rule applies: E-AddLit needs two literals, and the congruence rules need one of the operands to step, which `true` and `1` cannot. The term is **stuck**:

```
/-- A term is stuck when it is not a value and no rule applies to it. -/
def Stuck (t : Term) : Prop := ¬ Value t ∧ ∀ t', ¬ Step t t'
```

A stuck term is the formal model of a program that has gone wrong: it has not finished, but there is nothing it can do next. This is where an interpreter would crash, or where compiled code without checks would add a boolean to a number and continue with garbage. Proving that a particular term is stuck is a good first exercise in reasoning about an inductive relation:

```
/-- `true + 1` is stuck: it is not a value, and no rule has a conclusion of the
form `true + 1 → _`. -/
theorem stuck_example : Stuck (.add .tt (.lit 1)) := by
  constructor
  · -- Not a value: no constructor of `Value` builds an `add`.
    intro h
    cases h
  · -- No step: try every rule whose conclusion starts with `add`.
    intro t' h
    cases h with
    | addL h₁ => cases h₁    -- `true` would have to step: no rule for that
    | addR _ h₂ => cases h₂  -- `1` would have to step: no rule for that
```

`constructor` splits the conjunction into its two halves. For "not a value", we assume a proof `h` that `true + 1` is a value and run `cases h`: Lean examines every constructor of `Value`, finds that none of them produces an `add`, and closes the goal because there is no case left to consider. For "no step", we assume a step `h` and run `cases h`. This time two constructors could have produced a step from an `add`: `addL`, which needs a step from `true`, and `addR`, which needs a step from `1`. (`addLit` is ruled out because `true` is not a literal.) In each case, a second `cases` on the premise finds no rule that lets a literal step, and the goal is closed. This pattern, `cases` until no constructor fits, is how every "this cannot happen" argument in this chapter works.

The evaluator in Rust

The Rust crate has the same semantics in two forms, and the difference between them will matter for testing. The first, `successors`, is a transcription of the *relation*: it tries every rule independently, checks that rule's premises and returns every step that some rule allows.

```
/// Every step that the rules allow from `t`: one `if` per rule, each checking
/// exactly that rule's premises.
pub fn successors(t: &Term) -> Vec<Step> {
    let mut out = Vec::new();
    match t {
        Term::App(f, a) => {
            // E-AppL: f → f'
            for s in successors(f) {
                out.push(s.under(Rule::AppL, |f2| Term::App(Box::new(f2), boxed(a))));
            }
            // E-AppR: f is a value and a → a'
            if is_value(f) {
                for s in successors(a) {
                    out.push(
                        s.under(Rule::AppR, |a2| Term::App(boxed(f), Box::new(a2))),
                    );
                }
            }
            // E-Beta: f is a λ and a is a value
            if let (Term::Lam(_, body), true) = (&**f, is_value(a)) {
                out.push(Step::axiom(Rule::Beta, instantiate(body, a)));
            }
        }
        Term::If(c, then_branch, else_branch) => {
            // E-If: c → c'
            for s in successors(c) {
```

```
                out.push(s.under(Rule::If, |c2| {
                    Term::If(Box::new(c2), boxed(then_branch), boxed(else_branch))
                }));
            }
            // E-IfTrue and E-IfFalse
            match **c {
                Term::True => {
                    out.push(Step::axiom(Rule::IfTrue, (**then_branch).clone()))
                }
                Term::False => {
                    out.push(Step::axiom(Rule::IfFalse, (**else_branch).clone()))
                }
                _ => {}
            }
        }
        Term::Add(a, b) => {
            // E-AddL: a → a'
            for s in successors(a) {
                out.push(s.under(Rule::AddL, |a2| Term::Add(Box::new(a2), boxed(b))));
            }
            // E-AddR: a is a value and b → b'
            if is_value(a) {
                for s in successors(b) {
                    out.push(
                        s.under(Rule::AddR, |b2| Term::Add(boxed(a), Box::new(b2))),
                    );
                }
            }
            // E-AddLit: two literals
            if let (Term::Lit(m), Term::Lit(n)) = (&**a, &**b) {
                out.push(Step::axiom(Rule::AddLit, Term::Lit(m + n)));
            }
        }
        Term::IsZero(n) => {
            // E-IsZero: n → n'
            for s in successors(n) {
                out.push(s.under(Rule::IsZero, |n2| Term::IsZero(Box::new(n2))));
            }
            // E-IsZeroZero and E-IsZeroSucc
            match **n {
                Term::Lit(0) => out.push(Step::axiom(Rule::IsZeroZero, Term::True)),
                Term::Lit(_) => out.push(Step::axiom(Rule::IsZeroSucc, Term::False)),
                _ => {}
            }
        }
        // No rule has a variable or a value on its left-hand side.
        Term::Var(_) | Term::Lam(..) | Term::True | Term::False | Term::Lit(_) => {}
    }
    out
}
```

The second, `step`, is an *evaluator*: a function that returns the one step that call-by-value takes. It decides which rule to use by looking for the leftmost subterm that is not yet a value, which is the strategy written as code.

```
/// The evaluator: the step that call-by-value takes from `t`, or `None` if `t`
/// is a value or stuck. Each case first finds the leftmost subterm that is not
/// yet a value, which is what makes it a function.
pub fn step(t: &Term) -> Option<Step> {
    match t {
        Term::App(f, a) if !is_value(f) => {
            step(f).map(|s| s.under(Rule::AppL, |f2| Term::App(Box::new(f2), boxed(a))))
        }
        Term::App(f, a) if !is_value(a) => {
            step(a).map(|s| s.under(Rule::AppR, |a2| Term::App(boxed(f), Box::new(a2))))
        }
        Term::App(f, a) => match &**f {
            Term::Lam(_, body) => Some(Step::axiom(Rule::Beta, instantiate(body, a))),
```

```
            _ => None, // a value that is not a function, applied: stuck
        },
        Term::If(c, t1, t2) if !is_value(c) => step(c).map(|s| {
            s.under(Rule::If, |c2| Term::If(Box::new(c2), boxed(t1), boxed(t2)))
        }),
        Term::If(c, t1, t2) => match **c {
            Term::True => Some(Step::axiom(Rule::IfTrue, (**t1).clone())),
            Term::False => Some(Step::axiom(Rule::IfFalse, (**t2).clone())),
            _ => None, // a condition that is not a boolean: stuck
        },
        Term::Add(a, b) if !is_value(a) => {
            step(a).map(|s| s.under(Rule::AddL, |a2| Term::Add(Box::new(a2), boxed(b))))
        }
        Term::Add(a, b) if !is_value(b) => {
            step(b).map(|s| s.under(Rule::AddR, |b2| Term::Add(boxed(a), Box::new(b2))))
        }
        Term::Add(a, b) => match (&**a, &**b) {
            (Term::Lit(m), Term::Lit(n)) => {
                Some(Step::axiom(Rule::AddLit, Term::Lit(m + n)))
            }
            _ => None, // adding something that is not a number: stuck
        },
        Term::IsZero(n) if !is_value(n) => {
            step(n).map(|s| s.under(Rule::IsZero, |n2| Term::IsZero(Box::new(n2))))
        }
        Term::IsZero(n) => match **n {
            Term::Lit(0) => Some(Step::axiom(Rule::IsZeroZero, Term::True)),
            Term::Lit(_) => Some(Step::axiom(Rule::IsZeroSucc, Term::False)),
            _ => None,
        },
        // Variables (free ones: a closed term has none at the top) and values.
        Term::Var(_) | Term::Lam(..) | Term::True | Term::False | Term::Lit(_) => None,
    }
}
```

A `Step` records the derivation of the step along with its result: the congruence rules from the outside in, ending with the computation rule that did the work. A step is itself a derivation, a chain of rule instances, exactly like the Lean proof term `.appR .lam (.beta .lit)`, which says "E-AppR, whose premise is E-Beta".

```
/// One step `t → result`, with its derivation: the congruence rules from the
/// outside in, ending with the computation rule that did the work.
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct Step {
    pub derivation: Vec<Rule>,
    pub result: Term,
}
```

The relation is the specification and the function is the implementation. The theorem that connects them is determinism.

9.4 Determinism

Theorem (determinism). If $t \to t_1$ and $t \to t_2$, then $t_1 = t_2$.

The key lemma is that values are finished: no rule has a value on its left-hand side.

```
/-- Values do not step: no rule has a value on its left-hand side. -/
theorem Value.no_step {v t : Term} (hv : Value v) : ¬ Step v t := by
  intro h
  cases hv <;> cases h
```

`cases hv` considers each kind of value, `cases h` each rule that could make it step, and the combinator `<;>` runs the second tactic on every goal produced by the first. There are four kinds of values and no rule applies to any of them, so nothing is left.

The proof of determinism is by induction on the first step, h_1, followed by a case analysis of the second step, h_2. Here are the three cases for application:

```
/-- Evaluation is deterministic: a term has at most one next step. -/
theorem Step.deterministic {t t₁ t₂ : Term} (h₁ : Step t t₁) (h₂ : Step t t₂) :
    t₁ = t₂ := by
  induction h₁ generalizing t₂ with
  | appL h ih =>
    cases h₂ with
    | appL h' => rw [ih h']
    | appR hv _ => exact absurd h (hv.no_step)
    | beta _ => exact absurd h Value.lam.no_step
  | appR hv h ih =>
    cases h₂ with
    | appL h' => exact absurd h' hv.no_step
    | appR _ h' => rw [ih h']
    | beta hv' => exact absurd h hv'.no_step
  | beta hv =>
    cases h₂ with
    | appL h' => exact absurd h' Value.lam.no_step
    | appR _ h' => exact absurd h' hv.no_step
    | beta _ => rfl
```

`generalizing t₂` makes the induction hypothesis talk about any second step, not just the one we started with. Then each pair of rules falls into one of three patterns.

- **The same congruence rule twice** (`appL` with `appL`): both steps rewrote the same subterm, so by the induction hypothesis `ih h'` the two results of that subterm are equal, and `rw` rewrites one into the other, which closes the goal.
- **Two different rules** (`appL` with `appR`, `appL` with `beta`, and so on): one rule says that a subterm steps, the other that the same subterm is a value. `Value.no_step` says that both cannot hold, and `absurd h (...)` derives anything at all, the goal included, from a statement together with its negation. In the `appL` / `beta` cases the value is the λ itself (`Value.lam`).
- **The same computation rule twice** (`beta` with `beta`): both results are literally the same term, and `rfl` proves the equation.

The remaining nine cases, for `if`, `+` and `iszero`, follow the same three patterns. Whenever a pair of rules is not covered by the `with` clauses, it is because the two conclusions cannot have the same left-hand side, and `cases` has already discarded it.

Determinism has a satisfying consequence: a term has at most one result.

```
/-- A consequence of determinism: a term evaluates to at most one value. -/
theorem Steps.value_unique {t v₁ v₂ : Term} (h₁ : Steps t v₁) (h₂ : Steps t v₂)
    (hv₁ : Value v₁) (hv₂ : Value v₂) : v₁ = v₂ := by
  induction h₁ with
  | refl =>
    -- t = v₁ is a value, so the run to v₂ must be empty.
    cases h₂ with
    | refl => rfl
    | step h _ => exact absurd h hv₁.no_step
  | step h _ ih =>
    cases h₂ with
```

```
    | refl => exact absurd h hv₂.no_step
    | step h' rest =>
      -- Both runs take a first step; by determinism it is the same step.
      rw [Step.deterministic h h'] at *
      exact ih rest hv₁
```

The proof walks down the two runs together. If the first run is empty, it starts at the value v_1, from which the second run cannot step, so $v_1 = v_2$. If both runs take a first step, determinism says it is the same step; `rw [...] at *` replaces one intermediate term by the other in every hypothesis, and the induction hypothesis handles the rest of the runs.

Notice that determinism has nothing to do with types: it holds for every term, well typed or not. It is a property of the evaluation rules alone, and the premises "v_1 value" in the congruence rules are what make it true.

9.5 Canonical forms

We now turn to the theorems that involve types, and we start with a simple observation that the progress proof will need three times. If a *value* has type *bool*, which values could it be? Look at the typing rules: a λ has an arrow type, a literal has type *nat*, so the only candidates are *true* and *false*. That is a **canonical forms** lemma: it says what the values of each type look like.

```
/-- A value of type `bool` is `tt` or `ff`. -/
theorem canonical_bool {v : Term} (hv : Value v) (ht : HasType [] v .bool) :
    v = .tt ∨ v = .ff := by
  cases hv with
  | lam => cases ht        -- a λ has an arrow type, not bool
  | tt => left; rfl
  | ff => right; rfl
  | lit => cases ht        -- a literal has type nat, not bool

/-- A value of type `nat` is a literal. -/
theorem canonical_nat {v : Term} (hv : Value v) (ht : HasType [] v .nat) :
    ∃ n, v = .lit n := by
  cases hv with
  | lit => exact ⟨_, rfl⟩
  | _ => cases ht          -- every other value has a different type

/-- A value of type `σ → τ` is a λ whose parameter has type `σ`. -/
theorem canonical_arrow {v : Term} {σ τ : Ty} (hv : Value v)
    (ht : HasType [] v (.arrow σ τ)) : ∃ body, v = .lam σ body := by
  cases hv with
  | lam => cases ht; exact ⟨_, rfl⟩
  | _ => cases ht
```

Each proof starts with `cases hv`, one goal for each kind of value. In the impossible goals, `cases ht` finds that no rule could give, say, a λ the type `bool`: the conclusion of T-Lam is an arrow type, a different constructor, and the case is discarded. (`| _ =>` handles all the remaining kinds of value at once.) In `canonical_arrow`, `cases ht` does something more interesting: it unifies the T-Lam conclusion $\sigma' \to \tau'$, where σ' is the annotation of the λ, with the expected type $\sigma \to \tau$, and so learns that the annotation is σ.

9.6 Progress

Theorem (progress). If $\vdash t : \tau$ (in the empty context), then t is a value or there is a t' with $t \to t'$.

The context must be empty. A term with a free variable, such as $\#0 + 1$, is well typed in the context $[\mathit{nat}]$, but it is not a value and cannot step: it is waiting for a value to be substituted for $\#0$. Progress is a statement about *closed* programs, the ones you can actually run.

The proof is by induction on the typing derivation. In each case we know the last rule used, and the induction hypotheses tell us that each premise's term is a value or steps. Here it is in full:

```
/-- Progress: a closed, well-typed term is a value or can take a step. -/
theorem progress {t : Term} {τ : Ty} (ht : HasType [] t τ) :
    Value t ∨ ∃ t', Step t t' := by
  -- Induction on the derivation. The context is `[]` in every case, so Lean
  -- needs to remember that: `generalize` names it `Γ` with `hΓ : [] = Γ`.
  generalize hΓ : ([] : Ctx) = Γ at ht
  induction ht with
  | var h =>
    -- A variable in the empty context. `h : lookup [] i = some τ` computes to
    -- `none = some τ`, which no constructor of `=` can prove.
    subst hΓ
    cases h
  | lam _ _ => left; exact .lam
  | @app _ σ τ t₁ t₂ ht₁ _ ih₁ ih₂ =>
    right
    rcases ih₁ hΓ with hv₁ | ⟨t₁', h₁⟩
    · rcases ih₂ hΓ with hv₂ | ⟨t₂', h₂⟩
      · -- Both are values: by canonical forms, t₁ is a λ, and β applies.
        subst hΓ
        obtain ⟨body, rfl⟩ := canonical_arrow hv₁ ht₁
        exact ⟨_, .beta hv₂⟩
      · exact ⟨_, .appR hv₁ h₂⟩     -- t₁ is a value, t₂ steps
    · exact ⟨_, .appL h₁⟩           -- t₁ steps
  | tt => left; exact .tt
  | ff => left; exact .ff
  | @ite _ τ c t e hc _ _ ihc _ _ =>
    right
    rcases ihc hΓ with hvc | ⟨c', hc'⟩
    · -- The condition is a value of type bool: `tt` or `ff`.
      subst hΓ
      rcases canonical_bool hvc hc with rfl | rfl
      · exact ⟨_, .iteTrue⟩
      · exact ⟨_, .iteFalse⟩
    · exact ⟨_, .ite hc'⟩
  | lit => left; exact .lit
  | @add _ t₁ t₂ ht₁ ht₂ ih₁ ih₂ =>
    right
    rcases ih₁ hΓ with hv₁ | ⟨t₁', h₁⟩
    · rcases ih₂ hΓ with hv₂ | ⟨t₂', h₂⟩
      · -- Two values of type nat: two literals, and `addLit` applies.
        subst hΓ
        obtain ⟨m, rfl⟩ := canonical_nat hv₁ ht₁
        obtain ⟨n, rfl⟩ := canonical_nat hv₂ ht₂
        exact ⟨_, .addLit⟩
      · exact ⟨_, .addR hv₁ h₂⟩
    · exact ⟨_, .addL h₁⟩
  | @isZero _ t ht' ih =>
    right
    rcases ih hΓ with hv | ⟨t', h⟩
    · subst hΓ
      obtain ⟨n, rfl⟩ := canonical_nat hv ht'
      cases n with
      | zero => exact ⟨_, .isZeroZero⟩
      | succ n => exact ⟨_, .isZeroSucc⟩
    · exact ⟨_, .isZero h⟩
```

Let us go through it.

- `generalize hΓ : ([] : Ctx) = Γ at ht` replaces the empty context in `ht` by a variable `Γ` and records the equation `hΓ : [] = Γ`. Induction needs this: the rule T-Lam changes the context of its premise, so an induction over derivations must work for every context, and the equation carries the fact that ours is empty into every case. Whenever we need to know that the context is empty, `subst hΓ` replaces `Γ` by `[]` everywhere.
- **Variables.** A variable cannot be typed in the empty context: the premise `h : lookup [] i = some τ` computes to `none = some τ`, and `cases h` finds no way to prove that. The case is impossible, which is exactly why progress needs the context to be empty.
- **Values.** For `lam`, `tt`, `ff` and `lit` we choose the `left` side of the disjunction and give the `Value` constructor.
- **Application.** This is the interesting case. The pattern `@app _ σ τ t₁ t₂ ht₁ _ ih₁ ih₂` names the implicit arguments of the constructor (the `@` makes them visible), the typing derivation `ht₁` of the function, and the two induction hypotheses. We choose `right`: an application is never a value, so we must find a step. `rcases ih₁ hΓ with hv₁ | ⟨t₁', h₁⟩` splits on the induction hypothesis for the function: either it is a value (`hv₁`) or it steps to some `t₁'` (`h₁`). If it steps, E-AppL gives the step: `⟨_, .appL h₁⟩` is the pair of a witness (`_`, which Lean infers) and a proof that `t` steps to it. If it is a value, we split on the argument the same way. If the argument steps, E-AppR applies. If both are values, we finally need the canonical forms lemma: the function is a value of type $\sigma \to \tau$, so it is a λ. The pattern `⟨body, rfl⟩` in `obtain` does more than name the body: the `rfl` replaces `t₁` by `.lam σ body` everywhere, so that the goal becomes "`(λ:σ. body) t₂` steps", which E-Beta proves.
- **Conditionals.** The condition is a value or steps. If it steps, E-If. If it is a value, canonical forms says it is `tt` or `ff`, and `rcases ... with rfl | rfl` substitutes each possibility in turn; E-IfTrue and E-IfFalse finish.
- **Addition** is the application case again: left operand steps (E-AddL), or is a value and the right one steps (E-AddR), or both are values, hence literals by canonical forms, and E-AddLit applies.
- `iszero`: the argument steps (E-IsZero), or it is a literal n; `cases n` distinguishes 0 (E-IsZeroZero) from a successor (E-IsZeroSucc).

The shape of this proof is the same for every language: one canonical forms lemma per type with an elimination form, and evaluation rules that cover every combination of values the typing rules allow. Had we forgotten E-IsZeroSucc, the `succ` case would fail, and Lean would show us the stuck term. Progress is a **completeness check of the evaluator against the type system**.

9.7 Substitution, carefully

Preservation will need to know that E-Beta does not change the type, and E-Beta performs a substitution. So before preservation we need the **substitution lemma**: substituting a term of the right type for a variable preserves typing. On paper it is one line,

$$\text{if } \sigma :: \Gamma \vdash t : \tau \text{ and } \Gamma \vdash s : \sigma \text{ then } \Gamma \vdash t[s] : \tau,$$

but to prove it we first have to say exactly what $t[s]$ is, and that is where de Bruijn indices ask for care.

Renaming and shifting

Substitution must move terms under binders, and a term that moves under a binder must have its free variables renumbered: under one more λ, the variable that was #3 outside is #4 inside. The cleanest way to organize this is to treat a **renaming** as a function ρ from indices to indices, and to define how it acts on a term:

```
/-- Push a renaming under one binder: index 0 is the variable that the binder
introduces, and it stays 0; every other index `i + 1` refers to the variable `i`
of the outer scope, which is renamed by `ρ` and then pushed up by one again. -/
def liftRen (ρ : Nat → Nat) : Nat → Nat
  | 0 => 0
  | i + 1 => ρ i + 1

/-- Apply a renaming (a function from indices to indices) to every free variable
of a term. Under a λ the renaming is lifted. -/
def rename (ρ : Nat → Nat) : Term → Term
  | .var i => .var (ρ i)
  | .lam τ t => .lam τ (rename (liftRen ρ) t)
  | .app t₁ t₂ => .app (rename ρ t₁) (rename ρ t₂)
  | .tt => .tt
  | .ff => .ff
  | .ite c t e => .ite (rename ρ c) (rename ρ t) (rename ρ e)
  | .lit n => .lit n
  | .add t₁ t₂ => .add (rename ρ t₁) (rename ρ t₂)
  | .isZero t => .isZero (rename ρ t)

/-- Shifting: every free variable moves up by one. This is what a term needs
when it is carried under one more binder. -/
def shift (t : Term) : Term := rename (· + 1) t
```

The only subtle case is the λ. Inside the body, index 0 is the variable that this λ binds, which the renaming must leave alone; an index $i+1$ inside refers to the variable that was i outside, which the renaming sends to $\rho(i)$, and which inside the body must again be one higher, $\rho(i)+1$. That is `liftRen ρ`. With renaming in hand, **shifting** (moving a term under one more binder) is the renaming $i \mapsto i+1$:

```
/-- In `λ:nat. #0 + #1`, the `#0` is bound and the `#1` is free (it refers to
the variable just outside the λ, written `#0` out there). Shifting moves only
the free one. -/
example : shift (.lam .nat (.add (.var 0) (.var 1)))
    = .lam .nat (.add (.var 0) (.var 2)) := rfl
```

Substitution

A **substitution** is a function σ from indices to terms, and it acts on a term by replacing every free variable $\#i$ with $\sigma(i)$. Under a λ it must be lifted, in the same way as a renaming, with one extra twist: the term that replaces an outer variable now lives under one more binder, so it must be shifted.

```
/-- Push a substitution under one binder: index 0 is the new bound variable and
maps to itself; the term `σ i` that replaces outer variable `i` is shifted,
because it now lives under one more binder. -/
def liftSub (σ : Nat → Term) : Nat → Term
  | 0 => .var 0
  | i + 1 => shift (σ i)

/-- Simultaneous substitution: replace every free variable `var i` by `σ i`. -/
def subst (σ : Nat → Term) : Term → Term
```

```
  | .var i => σ i
  | .lam τ t => .lam τ (subst (liftSub σ) t)
  | .app t₁ t₂ => .app (subst σ t₁) (subst σ t₂)
  | .tt => .tt
  | .ff => .ff
  | .ite c t e => .ite (subst σ c) (subst σ t) (subst σ e)
  | .lit n => .lit n
  | .add t₁ t₂ => .add (subst σ t₁) (subst σ t₂)
  | .isZero t => .isZero (subst σ t)

/-- The substitution used by β-reduction: variable 0 becomes `s`, and every
other variable moves down by one, because the λ that bound variable 0 is gone. -/
def single (s : Term) : Nat → Term
  | 0 => s
  | i + 1 => .var i

/-- `instantiate body s` is the body of a λ with its bound variable replaced by
`s`; on paper we write it $t[s]$. -/
def instantiate (body s : Term) : Term := subst (single s) body
```

`single s` is the substitution performed by E-Beta. When the λ disappears, its bound variable $\#0$ becomes the argument s, and every other free variable of the body moves *down* by one, because there is one binder fewer between it and its own λ. The examples show all three effects:

```
/-- The body of `λ:nat. #0 + 1`, instantiated with `41`. -/
example : instantiate (.add (.var 0) (.lit 1)) (.lit 41)
    = .add (.lit 41) (.lit 1) := rfl

/-- Substituting under a binder. The body is `λ:nat. #1 #0` (inside it, `#1` is
the outer bound variable) and the argument is a free variable `#5`. The result
mentions `#6`: under the inner λ, the argument had to be shifted. -/
example : instantiate (.lam .nat (.app (.var 1) (.var 0))) (.var 5)
    = .lam .nat (.app (.var 6) (.var 0)) := rfl

/-- A free variable of the body that is *not* the one being replaced moves down
by one, because the λ that bound `#0` has disappeared. -/
example : instantiate (.add (.var 0) (.var 3)) (.lit 7)
    = .add (.lit 7) (.var 2) := rfl
```

In the second example the body is $\lambda : \mathit{nat}.\ \#1\ \#0$, whose $\#1$ is the outer bound variable, and we substitute the free variable $\#5$ for it. The result mentions $\#6$, not $\#5$: inside the inner λ, the variable that is $\#5$ outside is called $\#6$, and `liftSub` shifted it accordingly. Forget that shift and the substituted term would silently refer to the wrong variable, which is the de Bruijn version of variable capture. This is the kind of bug that a proof assistant will not let you make.

The Rust crate has the same functions, with renamings and substitutions as closures (file `code/rust/crates/safety/src/subst.rs`):

```
/// Replaces every free variable `Var(i)` of `t` by `sigma(i)`.
pub fn subst(t: &Term, sigma: &dyn Fn(usize) -> Term) -> Term {
    let go = |u: &Term| Box::new(subst(u, sigma));
    match t {
        Term::Var(i) => sigma(*i),
        Term::Lam(ty, body) => {
            // Under the binder, 0 maps to itself, and the replacement for an
            // outer variable is shifted: it now lives under one more λ.
            let lifted = |i: usize| {
                if i == 0 {
                    Term::Var(0)
                } else {
                    shift(&sigma(i - 1))
                }
```

```
            };
            Term::Lam(ty.clone(), Box::new(subst(body, &lifted)))
        }
        Term::App(f, a) => Term::App(go(f), go(a)),
        Term::True => Term::True,
        Term::False => Term::False,
        Term::If(c, t, e) => Term::If(go(c), go(t), go(e)),
        Term::Lit(n) => Term::Lit(*n),
        Term::Add(a, b) => Term::Add(go(a), go(b)),
        Term::IsZero(t) => Term::IsZero(go(t)),
    }
}

/// The body of a λ with its bound variable replaced by `s`: variable 0
/// becomes `s`, and every other variable moves down by one.
pub fn instantiate(body: &Term, s: &Term) -> Term {
    subst(body, &|i| if i == 0 { s.clone() } else { Term::Var(i - 1) })
}
```

The renaming lemma, and weakening

We prove the typing lemmas in the same order as the definitions: renaming first, then substitution, then the special case that β needs. A renaming preserves types provided it respects the contexts, that is, it sends each variable to a variable of the same type:

```
/-- Renaming preserves types, provided the renaming respects the contexts:
whenever `Γ` gives variable `i` type `τ`, `Δ` gives `ρ i` the same type. -/
theorem rename_typing {Γ Δ : Ctx} {ρ : Nat → Nat} {t : Term} {τ : Ty}
    (hρ : ∀ i τ, lookup Γ i = some τ → lookup Δ (ρ i) = some τ)
    (ht : HasType Γ t τ) : HasType Δ (rename ρ t) τ := by
  induction ht generalizing Δ ρ with
  | var h => exact .var (hρ _ _ h)
  | lam _ ih =>
    -- Under the binder, the lifted renaming respects the extended contexts.
    apply HasType.lam
    apply ih
    intro i τ' h
    cases i with
    | zero => exact h                    -- the bound variable: 0 ↦ 0
    | succ i => exact hρ i τ' h          -- an outer variable: i + 1 ↦ ρ i + 1
  | app _ _ ih₁ ih₂ => exact .app (ih₁ hρ) (ih₂ hρ)
  | tt => exact .tt
  | ff => exact .ff
  | ite _ _ _ ihc iht ihe => exact .ite (ihc hρ) (iht hρ) (ihe hρ)
  | lit => exact .lit
  | add _ _ ih₁ ih₂ => exact .add (ih₁ hρ) (ih₂ hρ)
  | isZero _ ih => exact .isZero (ih hρ)
```

The proof is by induction on the typing derivation, `generalizing Δ ρ` so that the induction hypotheses work for every target context and every renaming (we will need them for different ones). Most cases just rebuild the same rule from the induction hypotheses, like `.app (ih₁ hρ) (ih₂ hρ)`. Two cases do real work.

- **Variables.** The term is $\#i$ with `h : lookup Γ i = some τ`, the renamed term is $\#\rho(i)$, and the hypothesis `hρ` says precisely that `lookup Δ (ρ i) = some τ`. Rule T-Var does the rest.
- **λ.** After `apply HasType.lam`, the goal is to type the renamed body, `rename (liftRen ρ) t`, in the context `σ :: Δ`. The induction hypothesis `ih` can do that, provided we show that the lifted renaming respects the extended contexts `σ :: Γ` and `σ :: Δ`. That is the goal after `apply ih` and `intro i τ' h`, and we prove it by cases on the index. For `0`, the lookup in both contexts is the new entry `σ`, and `liftRen ρ 0` is `0`, so the hypothesis `h` already *is* the goal:

both compute to `some σ = some τ'`, and `exact h` succeeds because Lean compares types up to computation. For `i + 1`, the hypothesis computes to `lookup Γ i = some τ'` and the goal to `lookup Δ (ρ i) = some τ'`, which is `hρ` applied to `i`.

The most important instance is **weakening**: a term that is typed in Γ is still typed, after shifting, in a context with one more variable in front. It is the renaming lemma with $\rho(i) = i + 1$:

```
/-- Weakening: a term typed in `Γ` is still typed, after shifting, in a
context with one more variable in front. -/
theorem weakening {Γ : Ctx} {t : Term} {τ : Ty} (σ : Ty) (ht : HasType Γ t τ) :
    HasType (σ :: Γ) (shift t) τ :=
  rename_typing (fun _ _ h => h) ht
```

The side condition says that `lookup Γ i = some τ` implies `lookup (σ :: Γ) (i + 1) = some τ`, and the second statement computes to the first, so the identity function `fun _ _ h => h` proves it. This is the structural rule of weakening from Chapter 6, in de Bruijn clothing: adding an unused assumption does not invalidate a derivation, provided you renumber.

The substitution lemma

A substitution preserves types provided it replaces every variable by a term of that variable's type:

```
/-- Substitution preserves types, provided every variable is replaced by a term
of its own type: whenever `Γ` gives `i` type `τ`, the term `σ i` has type `τ`
in `Δ`. -/
theorem subst_typing {Γ Δ : Ctx} {σ : Nat → Term} {t : Term} {τ : Ty}
    (hσ : ∀ i τ, lookup Γ i = some τ → HasType Δ (σ i) τ)
    (ht : HasType Γ t τ) : HasType Δ (subst σ t) τ := by
  induction ht generalizing Δ σ with
  | var h => exact hσ _ _ h
  | @lam Γ α β t _ ih =>
    apply HasType.lam
    apply ih
    intro i τ' h
    cases i with
    | zero =>
      -- The bound variable is replaced by itself.
      exact .var h
    | succ i =>
      -- An outer variable is replaced by `shift (σ i)`, typed by weakening.
      exact weakening α (hσ i τ' h)
  | app _ _ ih₁ ih₂ => exact .app (ih₁ hσ) (ih₂ hσ)
  | tt => exact .tt
  | ff => exact .ff
  | ite _ _ _ ihc iht ihe => exact .ite (ihc hσ) (iht hσ) (ihe hσ)
  | lit => exact .lit
  | add _ _ ih₁ ih₂ => exact .add (ih₁ hσ) (ih₂ hσ)
  | isZero _ ih => exact .isZero (ih hσ)
```

The structure is identical to the renaming lemma. In the variable case, the variable $\#i$ is replaced by $\sigma(i)$, and the hypothesis `hσ` is exactly the statement that $\sigma(i)$ has the right type. In the λ case, the pattern `@lam Γ α β t _ ih` names the parameter type `α`, and we must show that the lifted substitution respects `α :: Γ` and `α :: Δ`. Index `0` is replaced by the variable $\#0$, which has type `α` by T-Var. Index `i + 1` is replaced by `shift (σ i)`, and here the substitution lemma uses the renaming lemma: `σ i` has type `τ'` in `Δ` by `hσ`, so its shift has type `τ'` in `α :: Δ` by weakening. This is the precise point where the two lemmas meet, and the reason renaming had to come first.

Finally, the form that β-reduction needs:

```
/-- The substitution lemma in the form β-reduction needs: if the body of a λ
has type `τ` when its parameter has type `σ`, and `s` has type `σ`, then the
instantiated body has type `τ`. -/
theorem instantiate_typing {Γ : Ctx} {σ τ : Ty} {body s : Term}
    (hbody : HasType (σ :: Γ) body τ) (hs : HasType Γ s σ) :
    HasType Γ (instantiate body s) τ := by
  apply subst_typing _ hbody
  intro i τ' h
  cases i with
  | zero =>
    -- Variable 0 has type σ in σ :: Γ: `h` computes to `some σ = some τ'`,
    -- and `cases` concludes that τ' is σ. Variable 0 is replaced by s : σ.
    cases h
    exact hs
  | succ i =>
    -- Variable i + 1 becomes var i, with the same type in Γ.
    exact .var h
```

`apply subst_typing _ hbody` reduces the goal to the side condition for the substitution `single s`, with `σ :: Γ` as the source context and `Γ` as the target. Index `0` has type `σ` and is replaced by `s`, which has type `σ` by `hs`; `cases h` turns `h : some σ = some τ'` into the knowledge that `τ'` is `σ`. Index `i + 1` is replaced by $\#i$, whose type in `Γ` is the type of $\#(i+1)$ in `σ :: Γ`, which is what `h` says.

That is the hard part of type safety, and it is all about variables: booleans and numbers appear only in routine cases. Add a construct to the language and these proofs get one more routine case each; change how binders work and this is the part you must redo.

9.8 Preservation

Theorem (preservation). If $\Gamma \vdash t : \tau$ and $t \to t'$, then $\Gamma \vdash t' : \tau$.

```
/-- Preservation: a step does not change the type of a term. -/
theorem preservation {Γ : Ctx} {t t' : Term} {τ : Ty}
    (ht : HasType Γ t τ) (hstep : Step t t') : HasType Γ t' τ := by
  -- Induction on the step; in each case, take the typing derivation apart.
  induction hstep generalizing τ with
  | appL _ ih =>
    cases ht with
    | app ht₁ ht₂ => exact .app (ih ht₁) ht₂
  | appR _ _ ih =>
    cases ht with
    | app ht₁ ht₂ => exact .app ht₁ (ih ht₂)
  | beta _ _ =>
    -- ht : HasType Γ (app (lam σ body) v) τ. Take it apart twice:
    cases ht with
    | app hlam hv =>
      cases hlam with
      | lam hbody =>
        -- hbody : HasType (σ :: Γ) body τ,  hv : HasType Γ v σ
        exact instantiate_typing hbody hv
  | ite _ ih =>
    cases ht with
    | ite hc ht₁ ht₂ => exact .ite (ih hc) ht₁ ht₂
  | iteTrue =>
    cases ht with
    | ite _ ht₁ _ => exact ht₁
  | iteFalse =>
    cases ht with
    | ite _ _ ht₂ => exact ht₂
  | addL _ ih =>
    cases ht with
    | add ht₁ ht₂ => exact .add (ih ht₁) ht₂
  | addR _ _ ih =>
```

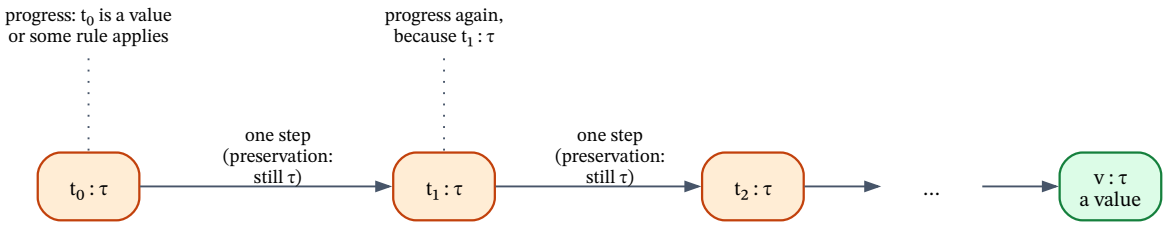
```
    cases ht with
    | add ht₁ ht₂ => exact .add ht₁ (ih ht₂)
  | addLit =>
    cases ht
    exact .lit
  | isZero _ ih =>
    cases ht with
    | isZero ht' => exact .isZero (ih ht')
  | isZeroZero =>
    cases ht
    exact .tt
  | isZeroSucc =>
    cases ht
    exact .ff
```

Textbooks usually prove preservation by induction on the typing derivation. We do it by induction on the *step*, `generalizing τ`, and then take the typing derivation apart with `cases`: the step tells us which rule fired, and in each case there is exactly one typing rule that could have typed the left-hand side. This "taking apart" is called **inversion**: from the fact that `app t₁ t₂` has type `τ`, `cases ht` concludes that there is a σ with $t_1 : \sigma \to \tau$ and $t_2 : \sigma$, because T-App is the only rule whose conclusion is an application.

- **Congruence rules** (`appL`, `appR`, `ite`, `addL`, `addR`, `isZero`): inversion gives the derivations of the subterms, the induction hypothesis `ih` says that the stepped subterm keeps its type, and the same typing rule reassembles the whole.
- **E-Beta**: here is where all the work of the previous section pays off. The term is $(\lambda : \sigma'.\, body)\, v$. Inversion of T-App gives `hlam`, a derivation for the λ with some type $\sigma \to \tau$, and `hv`, for the argument with type σ. Inversion of T-Lam on `hlam` gives `hbody : HasType (σ :: Γ) body τ` (and, silently, the fact that the annotation σ' is σ). The substitution lemma `instantiate_typing hbody hv` is exactly the conclusion.
- **E-IfTrue and E-IfFalse**: the result is one of the branches, and inversion of T-If gives its derivation, with the type `τ` of the whole `if`. This is where rule T-If's insistence on a single type for both branches is used.
- **E-AddLit, E-IsZeroZero, E-IsZeroSucc**: the result is a literal or a boolean, typed by an axiom. `cases ht` is still needed, to learn that `τ` is `nat` or `bool`.

9.9 Type safety

Progress talks about one state and preservation about one step. Safety talks about whole runs, and it follows from the two by an induction on the length of the run.



A run of a well-typed program: preservation keeps the type at every step, and progress, applied to each state in turn, says that the run never stops at a term that is not a value.

```
/-- Preservation for runs of any length. -/
theorem preservation_steps {t t' : Term} {τ : Ty}
    (ht : HasType [] t τ) (hsteps : Steps t t') : HasType [] t' τ := by
  induction hsteps with
  | refl => exact ht
  | step hstep _ ih => exact ih (preservation ht hstep)

/-- Type safety: a closed well-typed term never reaches a stuck state. -/
theorem safety {t t' : Term} {τ : Ty}
    (ht : HasType [] t τ) (hsteps : Steps t t') : ¬ Stuck t' := by
  intro ⟨hnotval, hnostep⟩
  -- t' is still well typed, so progress applies to it.
  rcases progress (preservation_steps ht hsteps) with hv | ⟨t'', h⟩
  · exact hnotval hv
  · exact hnostep t'' h
```

`preservation_steps` extends preservation to runs, by induction on the run: an empty run changes nothing, and a run that starts with a step keeps the type first by `preservation` and then by the induction hypothesis. `safety` then assumes that the end of the run is stuck, which `intro ⟨hnotval, hnostep⟩` splits into "not a value" and "no step". The term at the end of the run is still well typed, by `preservation_steps`, so progress applies to it: it is a value, contradicting `hnotval`, or it steps, contradicting `hnostep`.

That is the theorem Milner's slogan promised: a closed, well-typed term never reaches a stuck state, however many steps it takes. Specialized to one program, it is a one-line corollary:

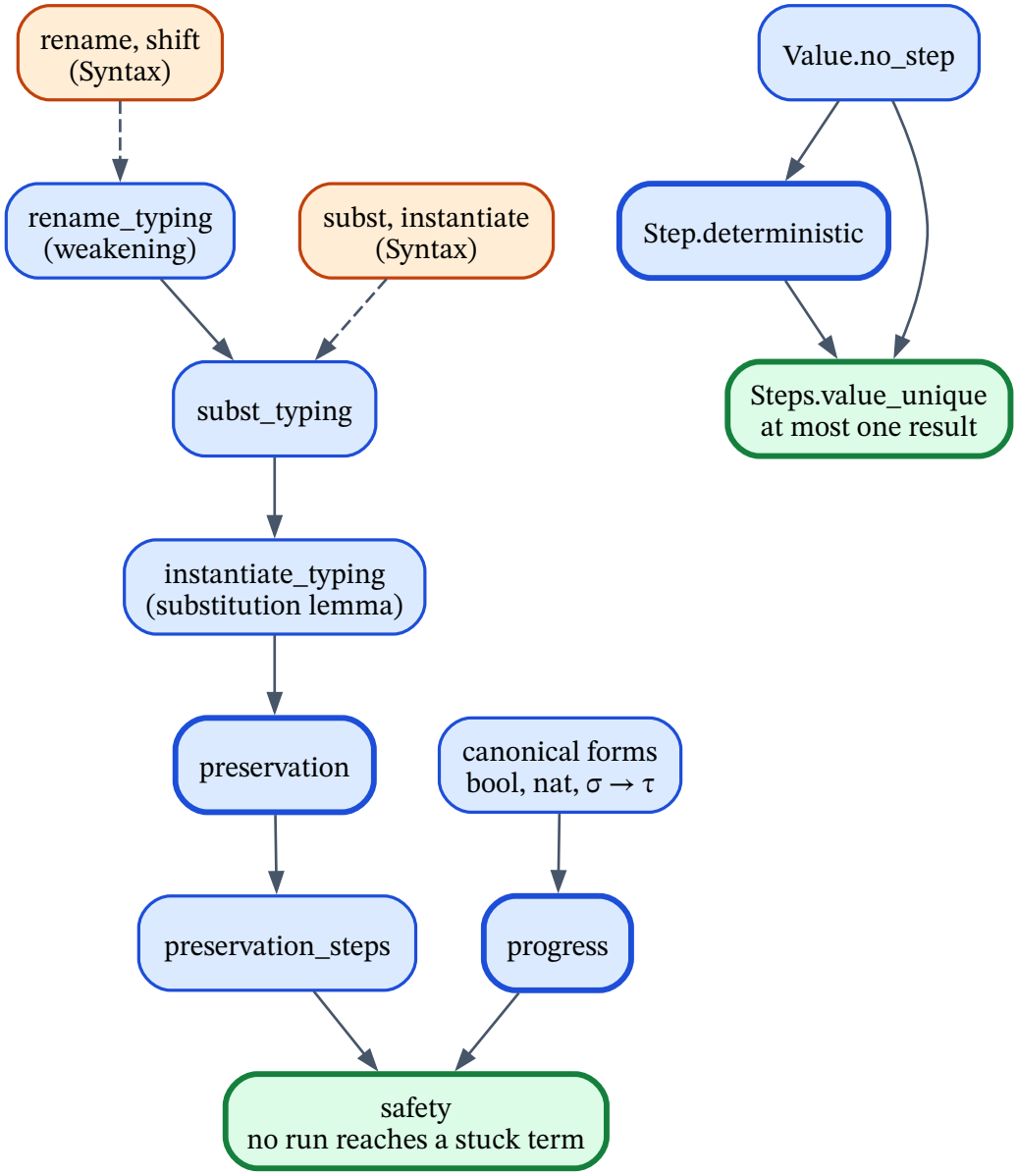
```
/-- For example: the program `(λ:nat. #0 + 1) 41` never gets stuck, whatever
it reduces to. The typing derivation comes from the checker. -/
theorem incr41_safe {t' : Term} (h : Steps incr41 t') : ¬ Stuck t' :=
  safety incr41_typed h
```

And one more check, of a different kind:

```
/-- info: 'Ch09.safety' does not depend on any axioms -/
#guard_msgs in
#print axioms safety
```

Chapter 1 used `#print axioms` to show that some proofs depend on axioms such as `propext` or `Classical.choice`. The proof of type safety depends on none: it is built from the rules of Lean's type theory alone, every case analysis is constructive, and the theorem holds in the strongest sense Lean has. (This is the reason for our own `lookup`: the definition of Lean's list indexing `Γ[i]?` happens to bring in the axiom `propext`. That would not make the proof wrong, but it is satisfying to know that it is not needed.)

The whole development fits in one picture. Arrows go from a lemma to the results that use it.



The structure of the proof: the substitution lemmas feed preservation, the canonical forms feed progress, and the two together give safety; determinism is a separate branch that does not involve types.

9.10 Testing the theorems in Rust

The Lean proof covers every term at once. The Rust crate `safety` takes the other road: it implements the same language and checks the same theorems on many concrete terms. The example `ch09_fuzz_safety`, in `code/rust/crates/safety/examples/ch09_fuzz_safety.rs`, does the whole tour; you run it with

```
cargo run -p safety --example ch09_fuzz_safety
```

Random numbers without a crate

The standard library has no random number generator, and the book's Rust code uses no other crates. A property-based test does not need good randomness, only *reproducible* variety, and a few lines of SplitMix64 provide it:

```rust
/// SplitMix64: a 64-bit state advanced by a constant, then scrambled.
/// Not for cryptography; good enough to drive a test generator.
#[derive(Clone, Debug)]
pub struct Rng {
    state: u64,
}

impl Rng {
    pub fn new(seed: u64) -> Rng {
        Rng { state: seed }
    }

    pub fn next_u64(&mut self) -> u64 {
        self.state = self.state.wrapping_add(0x9E37_79B9_7F4A_7C15);
        let mut z = self.state;
        z = (z ^ (z >> 30)).wrapping_mul(0xBF58_476D_1CE4_E5B9);
        z = (z ^ (z >> 27)).wrapping_mul(0x94D0_49BB_1331_11EB);
        z ^ (z >> 31)
    }

    /// A number in `0..n` (with a negligible bias for small `n`).
    pub fn below(&mut self, n: usize) -> usize {
        assert!(n > 0, "below(0)");
        (self.next_u64() % n as u64) as usize
    }

    /// An element of a non-empty slice.
    pub fn pick<'a, T>(&mut self, items: &'a [T]) -> &'a T {
        &items[self.below(items.len())]
    }
}
```

The same seed always produces the same sequence, so every failure can be replayed exactly. (Crates such as `proptest` and `quickcheck` package this whole approach, with better generators and automatic shrinking; we build a small version ourselves so that every piece is visible.)

Generating well-typed terms

A random term is almost never well typed: generate `if`, `+`, `true` and λ at random and you get `iszero (λ:bool. #0)` most of the time. Testing progress and preservation on such terms tests nothing, because the theorems only speak about well-typed terms. The generator must produce well-typed terms, and in large numbers.

The trick is to run the typing rules **backwards**. To generate a term of type τ, pick any rule whose conclusion can have type τ, and generate its premises recursively. To produce a nat, T-Lit, T-Add, T-If and T-App all qualify; for T-App, invent an argument type σ, then generate a function of type $\sigma \to \tau$ and an argument of type σ. To produce a $\sigma \to \tau$, use T-Lam: put σ in the context and generate a body of type τ, where T-Var may now pick the new variable.

```rust
/// A random term of type `ty` in context `ctx`, with roughly `size`
/// constructors, following the typing rules `rules` backwards.
pub fn gen_term(rng: &mut Rng, rules: Rules, ctx: &[Ty], ty: &Ty, size: usize) -> Term {
    // Collect the rules whose conclusion can have type `ty`.
    let mut choices = Vec::new();
```

```
    for (i, var_ty) in ctx.iter().enumerate() {
        if var_ty == ty {
            choices.push(Choice::Var(i)); // T-Var
        }
    }
    match ty {
        Ty::Bool => choices.extend([Choice::True, Choice::False]),
        Ty::Nat => choices.push(Choice::Lit),
        Ty::Arrow(..) => choices.push(Choice::Lam),
    }
    if size > 1 {
        // Rules with premises, when there is room for subterms. Applications
        // are listed twice: they create the β-redexes that make runs long.
        choices.extend([Choice::App, Choice::App, Choice::If]);
        match ty {
            Ty::Nat => choices.push(Choice::Add),
            Ty::Bool => choices.push(Choice::IsZero),
            Ty::Arrow(..) => {}
        }
    }
    let budget = size.saturating_sub(1);
    match *rng.pick(&choices) {
        Choice::Var(i) => Term::Var(i),
        Choice::True => Term::True,
        Choice::False => Term::False,
        Choice::Lit => Term::Lit(rng.below(10) as u64),
        Choice::Lam => {
            // T-Lam: ty is σ → τ; generate a body of type τ with σ in scope.
            let Ty::Arrow(param, result) = ty else {
                unreachable!()
            };
            let mut inner = vec![(**param).clone()];
            inner.extend_from_slice(ctx);
            lam(
                (**param).clone(),
                gen_term(rng, rules, &inner, result, budget),
            )
        }
        Choice::App => {
            // T-App: invent an argument type σ, then a function σ → ty and an
            // argument σ.
            let arg_ty = gen_type(rng, 1);
            let (left, right) = split(rng, budget);
            let f = gen_term(
                rng,
                rules,
                ctx,
                &Ty::arrow(arg_ty.clone(), ty.clone()),
                left,
            );
            app(f, gen_term(rng, rules, ctx, &arg_ty, right))
        }
        Choice::If => {
            // T-If: a boolean condition and two branches of type ty. The broken
            // rule lets the else branch have any type at all.
            let c = gen_term(rng, rules, ctx, &Ty::Bool, budget / 3);
            let t = gen_term(rng, rules, ctx, ty, budget / 3);
            let else_ty = match rules {
                Rules::Sound => ty.clone(),
                Rules::BrokenIf => gen_type(rng, 1),
            };
            ite(c, t, gen_term(rng, rules, ctx, &else_ty, budget / 3))
        }
        Choice::Add => {
            let (left, right) = split(rng, budget);
            let a = gen_term(rng, rules, ctx, &Ty::Nat, left);
            add(a, gen_term(rng, rules, ctx, &Ty::Nat, right))
        }
        Choice::IsZero => is_zero(gen_term(rng, rules, ctx, &Ty::Nat, budget)),
```

```
    }
}
```

The size budget shrinks at every rule with premises, which guarantees that generation stops. The terms are well typed by construction, but "by construction" is a claim, not a proof, so the checker verifies it for every term.

Checking a run

For each generated term, the harness runs it to a value and checks the three properties at every state along the way:

```
/// Runs `t`, which should have type `ty`, to a value, checking at every state:
/// the state has type `ty` (preservation; at the start, the generator's claim),
/// it is a value or the rules allow a step (progress), and the rules allow at
/// most one step, the one the evaluator takes (determinism).
/// Returns the number of steps taken.
pub fn check_run(
    rules: Rules,
    t: &Term,
    ty: &Ty,
    fuel: usize,
) -> Result<usize, Violation> {
    let found = type_of_with(rules, &[], t);
    if found.as_ref() != Ok(ty) {
        return Err(Violation::Generator {
            term: t.clone(),
            ty: ty.clone(),
            found,
        });
    }
    let mut current = t.clone();
    for steps_taken in 0..fuel {
        if is_value(&current) {
            return Ok(steps_taken);
        }
        let all = successors(&current);
        if all.is_empty() {
            return Err(Violation::Progress {
                state: current,
                ty: ty.clone(),
            });
        }
        if all.len() > 1 || step(&current).as_ref() != Some(&all[0]) {
            return Err(Violation::Determinism {
                state: current,
                steps: all,
            });
        }
        let next = all[0].result.clone();
        let found = type_of_with(rules, &[], &next);
        if found.as_ref() != Ok(ty) {
            let (before, after, ty) = (current, next, ty.clone());
            return Err(Violation::Preservation {
                before,
                after,
                ty,
                found,
            });
        }
        current = next;
    }
    Err(Violation::OutOfFuel { state: current })
}
```

Each check is a direct transcription of a theorem. **Preservation**: the type checker agrees that every new state has the original type. **Progress**: if the state is not a value, the relation allows at least one step. **Determinism**: it allows at most one, and it is the one the evaluator `step` chose. This last check is where having two versions of the semantics pays off: the relation `successors` is the specification, written rule by rule, and the function `step` is the implementation, written for efficiency. The test compares them on every state it visits.

Here is a run of ten thousand random programs:

```
== Property test: random well-typed terms ==
terms        : 10000 (seed 2024)
types        : 3389 bool, 3263 nat, 3348 function
largest term : 68 constructors
steps        : 32305 in total, longest run 19
progress     : held in all 42305 states visited
preservation : held for all 32305 steps
determinism  : one rule at most in every state, as the evaluator chose
```

Every theorem held in every state. The runs are short (a few steps on average): random well-typed programs tend to be shallow, which is a limitation of the generator worth keeping in mind.

What the type system rules out

The same crate can generate terms with no regard for types. Closed random terms of up to twelve constructors give a feeling for what typing is for:

```
== Random closed terms, ignoring types ==
well typed   : 3380
ill typed    : 6620
   stuck              : 5299
   reached a value    : 1321
   ran out of fuel    : 0
```

Of the terms that the checker rejects, about four in five really do get stuck. The rest run to a value without trouble: `if true then 1 else false` is rejected because its branches have different types, yet it evaluates to `1`. A type system is **conservative**: it rejects every program that could go wrong, and some that would not have. Four hand-picked examples:

```
== Ill-typed terms ==
true + 1
   rejected: the left operand of `+` should be nat, found bool
   it is stuck at true + 1 after 0 steps
(λ:nat. #0 + 1) true
   rejected: the argument should be nat, found bool
   it is stuck at true + 1 after 1 step
if true then 1 else false
   rejected: the branches of `if` differ: nat and bool
   it reaches the value 1 in 1 step
(λ:bool. #0 #0) (λ:bool. #0 #0)
   rejected: applying a non-function of type bool
   it is still running after 100 steps
```

The second one takes a perfectly good β-step before it gets stuck at `true + 1`: in an ill-typed program the error may appear far from its cause, which is exactly what preservation prevents for well-typed ones. The last is the Ω of Chapter 7, which runs forever. It is ill typed too; as Chapter 8 said, every

well-typed term of the simply typed λ-calculus terminates. Our safety theorem does not need that fact, as we will see at the end of the chapter.

Breaking a rule on purpose

A test suite that never fails says little about its own power, so let us break the type system and see whether the tests notice. The crate's `Rules::BrokenIf` changes one rule: T-If takes the type of its `then` branch and ignores the `else` branch entirely. The generator follows the broken rule too, so it produces terms like `if c then 1 else true`, which the broken checker calls `nat`. Here is what the harness says:

```
== The same test, with the broken rule for `if` ==
term #3 (29 constructors) violates preservation
shrunk to    : if false then 0 else false : nat
   preservation fails: if false then 0 else false : nat  →  false : bool
and the counterexample from Safety.lean:
   (if false then 1 else true) + 2  : nat
→  true + 2  : ill typed: the left operand of `+` should be nat, found bool
      by E-AddL(E-IfFalse)
   (stuck)
```

Term number 3 (counting from 0) already violates preservation. The raw counterexample has 29 constructors, which is not very informative, so the harness **shrinks** it: it repeatedly tries smaller candidates (a subterm in place of the whole, a constant in place of a subterm) and keeps any candidate that still has a type under the broken rules and still violates a theorem.

```rust
/// Shrinks a failing term: repeatedly replaces it by a smaller candidate that
/// still has a type (under `rules`) and still violates a theorem.
pub fn shrink(rules: Rules, term: &Term, fuel: usize) -> (Term, Ty, Violation) {
    let ty = type_of_with(rules, &[], term).expect("a failing term has a type");
    let violation = check_run(rules, term, &ty, fuel).expect_err("the term fails");
    let mut best = (term.clone(), ty, violation);
    'improve: loop {
        for candidate in shrink_candidates(&best.0) {
            // Only candidates that are still "well typed" count...
            let Ok(ty) = type_of_with(rules, &[], &candidate) else {
                continue;
            };
            // ...and that still break a theorem.
            if let Err(violation) = check_run(rules, &candidate, &ty, fuel) {
                best = (candidate, ty, violation);
                continue 'improve;
            }
        }
        return best;
    }
}
```

The result, `if false then 0 else false : nat`, is as small as a counterexample can be: one step later it is `false`, a `bool`. Preservation fails, and with it safety: the counterexample that the Lean file uses, `(if false then 1 else true) + 2`, steps to `true + 2`, which is stuck.

The Lean side can say the same thing as a theorem. `BadHasType` is `HasType` with the broken rule:

```
/-- A *broken* type system: identical to `HasType`, except that the rule for
`if` only looks at the `then` branch. -/
inductive BadHasType : Ctx → Term → Ty → Prop
  | var {Γ : Ctx} {i : Nat} {τ : Ty} :
      lookup Γ i = some τ → BadHasType Γ (.var i) τ
  | lam {Γ : Ctx} {σ τ : Ty} {t : Term} :
      BadHasType (σ :: Γ) t τ → BadHasType Γ (.lam σ t) (.arrow σ τ)
```

```
  | app {Γ : Ctx} {σ τ : Ty} {t₁ t₂ : Term} :
      BadHasType Γ t₁ (.arrow σ τ) → BadHasType Γ t₂ σ →
      BadHasType Γ (.app t₁ t₂) τ
  | tt {Γ : Ctx} : BadHasType Γ .tt .bool
  | ff {Γ : Ctx} : BadHasType Γ .ff .bool
  | ite {Γ : Ctx} {τ τ' : Ty} {c t e : Term} :          -- τ' is ignored!
      BadHasType Γ c .bool → BadHasType Γ t τ → BadHasType Γ e τ' →
      BadHasType Γ (.ite c t e) τ
  | lit {Γ : Ctx} {n : Nat} : BadHasType Γ (.lit n) .nat
  | add {Γ : Ctx} {t₁ t₂ : Term} :
      BadHasType Γ t₁ .nat → BadHasType Γ t₂ .nat →
      BadHasType Γ (.add t₁ t₂) .nat
  | isZero {Γ : Ctx} {t : Term} :
      BadHasType Γ t .nat → BadHasType Γ (.isZero t) .bool
```

and the broken system is provably unsafe:

```
/-- The broken system is not safe: `(if false then 1 else true) + 2` is
"well typed" in it, and after one step it is stuck. -/
theorem bad_typing_unsafe :
    ∃ t t' τ, BadHasType [] t τ ∧ Steps t t' ∧ Stuck t' := by
  refine ⟨.add (.ite .ff (.lit 1) .tt) (.lit 2),   -- the program
          .add .tt (.lit 2),                        -- where it ends up
          .nat, ?_, ?_, ?_⟩
  · -- "Well typed": the `then` branch is a nat, the `else` branch is ignored.
    exact .add (.ite .ff .lit .tt) .lit
  · -- One step: the `if` reduces to its `else` branch.
    exact .step (.addL .iteFalse) .refl
  · -- `true + 2` is stuck, for the same reason as `true + 1`.
    constructor
    · intro h; cases h
    · intro t'' h
      cases h with
      | addL h₁ => cases h₁
      | addR _ h₂ => cases h₂
```

`refine` fills in the three witnesses and leaves three goals, marked `?_`, each proved as before. And if you copy the proof of `preservation` and replace `HasType` by `BadHasType`, it breaks in exactly one place: in the `iteFalse` case, inversion gives the `else` branch a type `τ'` unrelated to `τ`, and `exact ht₂` is rejected. The proof does not just say "unsafe"; it points at the rule.

Tests and proofs, compared

We now have two kinds of evidence for the same theorems, and it is worth being precise about what each gives.

	Property tests (Rust)	Proof (Lean)
Covers	the terms generated	every term
Checks	the Rust code that runs	a model of the language
On failure	a concrete, shrunk counterexample	a goal that cannot be closed
Cost	minutes to write, seconds to run	hours to write, then free
Explains	nothing: it found a bug	why: which lemma, which case

The first two rows are the important ones. The proof covers *all* terms; the tests cover those the generator happens to produce, and our generator has blind spots (short runs, small types), so a bug that only shows up in deep terms could escape ten thousand tests. On the other hand, the proof is

about the Lean definitions, while the program that runs is the Rust crate, and nothing in the Lean file prevents the Rust `step` function from having a bug of its own. That is why the harness checks the Rust evaluator against a rule-by-rule transcription of the relation. Proofs are about models, tests are about code, and a careful project uses both; Chapter 31 tests a Rust implementation against a verified Lean model.

9.11 Putting it together: one program, three views

Take the program $twice\ (\lambda : nat.\ \#0 + 2)\ 3$, which applies "add two" twice to three. In Rust, the evaluator shows the run, with the type of every intermediate term and the derivation of every step:

```
let add_two = lam(Ty::Nat, add(Term::Var(0), Term::Lit(2)));
let program = app(app(twice(), add_two), Term::Lit(3));
println!("== One run, step by step ==");
show_run(Rules::Sound, &program);
```

```
== One run, step by step ==
   (λ:(nat → nat). λ:nat. #1 (#1 #0)) (λ:nat. #0 + 2) 3  : nat
→  (λ:nat. (λ:nat. #0 + 2) ((λ:nat. #0 + 2) #0)) 3  : nat
      by E-AppL(E-Beta)
→  (λ:nat. #0 + 2) ((λ:nat. #0 + 2) 3)  : nat
      by E-Beta
→  (λ:nat. #0 + 2) (3 + 2)  : nat
      by E-AppR(E-Beta)
→  (λ:nat. #0 + 2) 5  : nat
      by E-AppR(E-AddLit)
→  5 + 2  : nat
      by E-Beta
→  7  : nat
      by E-AddLit
   (a value)
```

Read the derivations. In the first step, `E-AppL(E-Beta)`, the function part `twice (λ:nat. #0 + 2)` is itself a β-redex, so E-AppL descends into it and E-Beta fires there, replacing both occurrences of $\#1$ by the argument. The second step is a plain β at the top. The third, `E-AppR(E-Beta)`, evaluates the argument of the outer call first, as call-by-value demands, and so on down to `7`. Every line has type `nat`: preservation, visibly.

In Lean, the same program, its type, its run and its result:

```
/-- `λ (x : nat). x + 2` -/
def addTwo : Term := .lam .nat (.add (.var 0) (.lit 2))

/-- `twice (λ (x : nat). x + 2) 3` -/
def program : Term := .app (.app twice addTwo) (.lit 3)

/-- info: (λ:(nat → nat). λ:nat. #1 (#1 #0)) (λ:nat. #0 + 2) 3 -/
#guard_msgs in
#eval program

theorem program_typed : HasType [] program .nat := typeOf_sound rfl
```

```
/-- The run of `program`, one rule instance per step. Lean computes each
intermediate term itself (by unfolding `instantiate`), and checks that the
rules apply. -/
theorem program_runs : Steps program (.lit 7) :=
  .step (.appL (.beta .lam)) <|   -- twice addTwo 3  →  (λ:nat. addTwo (addTwo #0)) 3
```

```
  .step (.beta .lit) <|           --                → addTwo (addTwo 3)
  .step (.appR .lam (.beta .lit)) <| --             → addTwo (3 + 2)
  .step (.appR .lam .addLit) <|   --                → addTwo 5
  .step (.beta .lit) <|           --                → 5 + 2
  .step .addLit <|                --                → 7
  .refl

/-- Safety and determinism together: `program` never gets stuck, and `7` is the
only value it can produce. -/
theorem program_result {v : Term} (h : Steps program v) (hv : Value v) :
    v = .lit 7 :=
  Steps.value_unique h program_runs hv .lit
```

The proof of `program_runs` is the Rust trace, written as a proof term. The derivation `E-AppL(E-Beta)` is `.appL (.beta .lam)`: the argument of `.beta` is the proof that the argument of the call, `addTwo`, is a value. The operator `<|` applies the function on its left to everything on its right, and saves us six levels of nested parentheses: `f <| g <| x` means `f (g x)`. Lean computes each intermediate term itself, by unfolding `instantiate`, and checks that each rule applies; if the trace were wrong in any detail, the proof would fail. Finally, `program_result` combines the run with determinism: since `program` reaches `7` and a term has at most one value, `7` is the only value it can ever produce.

9.12 What "going wrong" means for Rust

Our language goes wrong in one way: it gets stuck. Real languages are more complicated, and it is worth asking what the analogue of a stuck state is for Rust, and what the analogue of the safety theorem promises.

Rust has no official formal semantics, so there is no theorem to quote. But the language reference does say what counts as going wrong: **undefined behavior**. Reading freed or uninitialized memory, a data race, producing an invalid value (a `bool` that is neither `true` nor `false`, a null or dangling reference) are all undefined behavior. These are Rust's stuck states: situations to which the language assigns no meaning at all. Safe Rust's promise, the analogue of our safety theorem, is that code without `unsafe` blocks cannot cause undefined behavior, provided the `unsafe` code inside the libraries it uses is correct. (For a formal core of Rust and several standard library types, that promise has been proved by the RustBelt project, with a technique quite different from progress and preservation; Chapter 32 is about it.)

Equally important is what does *not* count as going wrong.

- **Panics.** Indexing out of bounds, `unwrap` on `None`, integer overflow in a debug build, an explicit `panic!`: these are all defined behavior (the thread unwinds, running destructors, or the process aborts). In the vocabulary of this chapter, a panic is not a stuck state but a transition to an explicit *error* state, like the `error` term that textbooks add to a language with exceptions; progress then says "a value, a step, or an error".
- **Non-termination.** `loop {}` never gets stuck: it takes steps forever, and a diverging program is perfectly safe. Our safety theorem would survive the addition of general recursion to the language (with one more case in each proof); strong normalization is a separate, stronger property, which Rust does not have.

- **Leaks.** Forgetting to free memory (with `mem::forget`, or a cycle of `Rc` pointers) is safe. Chapter 24 explains why Rust's designers had to accept this.
- **Deadlocks** and **logic errors**. Two threads waiting for each other's locks are stuck in an informal sense, but every thread is in a well-defined state. And a program that computes the wrong answer is, as far as the type system is concerned, a correct program for a different specification.

So "well-typed programs cannot go wrong" is true for safe Rust, with "wrong" meaning undefined behavior. It is a strong and valuable guarantee, but a narrow one, and knowing its edges is part of using the language well. To say more about a program you need a stronger type (Chapter 30) or a proof about the program itself (Chapter 31).

9.13 Common confusions

"**Progress means the program terminates.**" Progress says that a well-typed program that has not finished can take *one more step*. A program can take one more step forever. Termination is a separate property: our language happens to have it, Rust does not, and safety does not depend on it.

"**Preservation means the term does not change.**" The term changes at every step; its *type* does not. `5 + 2` and `7` are different terms of the same type.

"**Safety means the result is correct.**" Safety means the result is a value of the promised type. `λ:nat. #0 + 1` and `λ:nat. #0 + 2` have the same type, and the type system cannot tell them apart.

"**The tests passed ten thousand times, so the theorem holds.**" The tests passed for ten thousand terms drawn from a particular distribution. The theorem holds for all terms because of the proof. The two statements are different, and the second one does not follow from the first. (The converse also fails: the proof does not show that the Rust code is correct.)

"**De Bruijn indices are an implementation trick.**" They are a design decision about the *mathematics*: they make α-equivalent terms equal, which removes a whole family of side conditions about fresh names from the substitution lemma.

9.14 Summary

- A small-step operational semantics is an inductive relation $t \to t'$ with computation rules (E-Beta, E-IfTrue, ...) and congruence rules (E-AppL, E-AppR, ...). The premises "value" in the congruence rules fix the evaluation order and make evaluation deterministic.
- A stuck term is neither a value nor able to step. Type safety says that a closed, well-typed term never reaches one.
- **Progress** is proved by induction on the typing derivation, with one canonical forms lemma per type ("a value of type bool is true or false").
- **Preservation** is proved by induction on the step with inversion of the typing derivation; its only hard case is E-Beta, which needs the **substitution lemma**, which in turn needs the **renaming lemma** (weakening is its special case).

- With de Bruijn indices, renaming and substitution are functions on indices, lifted under binders, and the lemmas about them are short inductions. All of it is checked by Lean, with no axioms.
- Determinism is independent of types; with it, a term has at most one value.
- Property-based tests check the same theorems on concrete terms, generated by running the typing rules backwards. They find bugs quickly and give concrete, shrinkable counterexamples; proofs cover all terms but are about a model.
- For Rust, "going wrong" means undefined behavior. Safe Rust rules it out; it does not rule out panics, non-termination, leaks or deadlocks.

9.15 Exercises

1. Write, on paper, the full derivation tree of the step $(\lambda : \mathit{nat}.\, \#0 + 2)\,(3 + 2) \to (\lambda : \mathit{nat}.\, \#0 + 2)\,5$, and the typing derivations of the terms on both sides. *Hint:* the step is E-AppR, whose premises are "the λ is a value" and the step $3 + 2 \to 5$, which is E-AddLit. Compare with the Lean term `.appR .lam .addLit` in `program_runs`.
2. Suppose we dropped the premise "v_1 value" from E-AppR. Find a term with two different next steps, so that determinism fails. Does safety still hold? *Hint:* take an application in which both the function and the argument can step, such as $((\lambda : \mathit{nat}.\, \lambda : \mathit{nat}.\, \#0)\,1)\,(2 + 3)$. For safety, look at which lemmas the proof of preservation uses: none of them mentions evaluation order.
3. Prove in Lean that a run can be continued inside a context: `theorem Steps.addL {t t' u : Term} (h : Steps t t') : Steps (.add t u) (.add t' u)`. Add it to a scratch copy of `Semantics.lean`. *Hint:* induction on `h`. The `refl` case is `.refl`; in the `step` case, combine `.addL hstep` with the induction hypothesis.
4. Prove that types are unique: `theorem HasType.unique {Γ t τ₁ τ₂} (h₁ : HasType Γ t τ₁) (h₂ : HasType Γ t τ₂) : τ₁ = τ₂`. *Hint:* induction on `h₁` generalizing `τ₂`, then `cases h₂`. In the `var` case, rewrite one `lookup` equation with the other and use `Option.some.inj`. In the `app` case, the induction hypothesis for the function gives an equation between two arrow types, and `cases` on it (or `Ty.arrow.inj`) gives the equation between their results.
5. Break a different rule in the Rust crate: make T-IsZero return `nat` instead of `bool` (add a variant to `Rules`). Which theorem does the harness report as violated, and what does the shrunk counterexample look like? Then state and prove the corresponding `bad_..._unsafe` theorem in Lean. *Hint:* `iszero 0` would then have type `nat` but step to `true`. Expect a preservation failure on a term of two or three constructors; wrapping it in `+ 1` gives a stuck term.
6. Extend the language with a boolean negation `not t`, with rules $\mathit{not}\ \mathit{true} \to \mathit{false}$, $\mathit{not}\ \mathit{false} \to \mathit{true}$ and a congruence rule, in both Lean and Rust. Update every definition and proof until everything compiles and the harness passes again. *Hint:* Lean will tell you exactly which cases are missing: each `match` and each induction needs one more case, and progress needs `canonical_bool` once more. Count how many of the new cases are routine.